

Systemes informatiques

Hugo Honorez
Matthias Giangreco
Adam Dikri

Décembre 2025

Table des matières

1	Introduction	2
2	Approfondissement de l'implémentation	3
2.1	Architecture et choix de conception	3
2.2	Compilation et automatisation	3
2.3	Cœur algorithmique : la suite de Conway	4
2.3.1	Gestion de la mémoire dynamique	4
2.4	Début du programme et options	5
2.4.1	Implémentation de l'option <code>--only-longest</code>	5
2.4.2	Implémentation de l'option <code>--debug</code>	6
2.4.3	Écriture du résultat	6
2.5	Gestion des LEDs	6
3	Gestion de la mémoire	8
3.1	Approche préventive	8
3.2	Validation avec Valgrind	9
3.3	Conclusion sur la fiabilité	9
4	Implémentation hardware	10
4.1	Description du montage électronique	10
4.2	Justification technique et contrôle	11
4.2.1	Configuration en anode commune	11
4.2.2	Dimensionnement des résistances	11
4.3	Correspondance des broches	11
5	Évaluation quantitative et analyse des performances	12
5.1	Environnement et protocole de test	12
5.1.1	Environnement	12
5.1.2	Méthodologie de reproduction	12
5.2	Analyse des résultats	13
5.2.1	Comparaison des temps d'exécution (Speedup)	13
5.2.2	Comparaison de l'utilisation de la mémoire (RAM)	14
5.2.3	Données détaillées	14
5.3	Conclusion du benchmark	15
6	Conclusion	16

Chapitre 1

Introduction

Dans ce projet, nous nous intéressons à la suite mathématique de Conway, dite « look-and-say », dont la particularité est de générer chaque nouveau terme par la description des chiffres du précédent. Le défi majeur de cette suite réside dans sa croissance exponentielle : la longueur des termes double approximativement à chaque itération, ce qui entraîne rapidement une explosion du temps de calcul et une saturation potentielle de la mémoire vive (RAM), rendant la tâche critique pour des systèmes à ressources limitées.

Pour répondre à cette problématique, nous avons conçu un programme en langage C optimisé, destiné à être exécuté sur un système embarqué (Raspberry Pi) et capable d'interagir avec un circuit de LEDs pour visualiser l'état des calculs. L'objectif principal de cette étude est de valider l'efficacité d'une approche compilée et légère face à une implémentation interprétée en Python, en démontrant sa capacité à maintenir des performances élevées malgré la complexité algorithmique croissante.

Chapitre 2

Approfondissement de l'implémentation

2.1 Architecture et choix de conception

Pour garantir la maintenabilité et la lisibilité du projet, nous avons opté pour une architecture modulaire. Le code est segmenté en fichiers d'en-tête (`.h`) et sources (`.c`), définissant des interfaces claires pour chaque responsabilité :

- **Gestion des options et des inputs/outputs (`main.c`)** : Chef d'orchestre du projet, il s'occupe de la gestion des options ainsi que des fichiers et du lancement des calculs.
- **Gestion des LEDs (`led.c` / `led.h`)** : Encapsule les appels systèmes à la librairie `libgpiod`. Elle expose des fonctions pour initialiser les GPIOs et changer les couleurs de ces mêmes LEDs.
- **Calcul (`calcul.c` / `calcul.h`)** : Contient la logique mathématique pure de la suite de Conway, indépendante de l'affichage ou du matériel.
- **Débugage (`debug.h`)** : Contient une fonction permettant un affichage conditionnel des messages selon un niveau choisi par l'utilisateur lors de l'utilisation de l'option de debug.

2.2 Compilation et automatisation

Afin de simplifier le processus de compilation, un fichier **Makefile** a été mis en place. Il automatise la génération de l'exécutable et la gestion des dépendances.

Listing 2.1 – Extrait du Makefile

```
CC = gcc
TARGET = main
OBJS = main.o calcul.o led.o -lgpiod

all: $(TARGET)

$(TARGET): $(OBJS)
    $(CC) $(OBJS) -o $(TARGET)
```

```
clean:
    rm -f $(TARGET)
```

2.3 Cœur algorithmique : la suite de Conway

La librairie de calcul repose sur deux fonctions principales :

1. **solve** : Fonction centrale qui itère sur la suite. Elle prend en entrée la chaîne initiale et le nombre d'itérations, et retourne le pointeur vers la chaîne finale et donc le n-ième terme recherché.
2. **count_digit** : Fonction comptant le nombre de caractères uniques (chiffres distincts) dans une chaîne.

Attention, `count_digit` est invoquée uniquement si l'option `--only-longest` est active, afin d'éviter des calculs superflus dans le cas d'une exécution standard, dans le cas contraire seule la fonction `solve` est appelée.

2.3.1 Gestion de la mémoire dynamique

La croissance de la suite étant exponentielle, la gestion de la RAM est critique. Dans la fonction `solve`, nous utilisons une stratégie de réallocation dynamique grâce aux fonctions `malloc()`, `realloc()`, `free()`.

Pour chaque itération, si la taille du buffer est trop petite, on effectue un `realloc()` pour anticiper le pire cas (où chaque chiffre est différent du précédent) de cette manière :

$$\text{taille_requis} = (\text{longueur_actuelle} \times 2) + 32$$

Si la taille de l'espace mémoire actuellement utilisé est plus grande que le double de la taille de l'itéré actuel + 32, alors on n'effectue pas de nouvelle allocation mémoire et on travaille dans l'espace déjà réservé. La marge de 32 octets garantit la présence du caractère de fin de chaîne `'\0'` et prévient tout débordement mémoire.

De plus, plutôt que de copier les chaînes à chaque itération (opération coûteuse), nous effectuons un simple échange de pointeurs entre le buffer de lecture et le buffer d'écriture :

Listing 2.2 – Logique de gestion mémoire et swap

```
if (new_value_alloc < needed_alloc) {
    char *tmp = realloc(new_value, needed_alloc);
    if (tmp == NULL) { // Gestion erreur // }
    new_value = tmp;
    new_value_alloc = needed_alloc;
}

char *tmp = current_value;
current_value = new_value;
new_value = tmp;
```

Cette méthode permet de réutiliser l'espace mémoire déjà alloué tant qu'il est suffisant, limitant ainsi les appels systèmes à `realloc()`.

2.4 Début du programme et options

La fonction `main` analyse les arguments de la ligne de commande pour configurer l'exécution. Les options possibles sont les suivantes :

- `--debug <N>` : Définit le niveau de débogage (0 à 2). La fonction `DEBUG` comparera le niveau demandé avec le niveau de priorité de chaque message de débog et affichera les messages qui correspondent au niveau demandé par l'utilisateur.
- `--input <file>` : Spécifie un fichier d'entrée. Le format attendu par ligne est strict : "chaîne_depart nombre_iterations". En l'absence de ce paramètre, la lecture se fait sur l'entrée standard (`stdin`).
- `--output <file>` : Redirige la sortie vers un fichier. Par défaut, le résultat est affiché sur la sortie standard (`stdout`).
- `--only-longest` : Mode spécifique n'affichant que le résultat possédant la plus grande diversité de chiffres et la plus grande longueur.

2.4.1 Implémentation de l'option `--only-longest`

Par défaut, le programme fonctionne en flux continu (calcul puis affichage immédiat). L'option `only-longest` impose une contrainte : tous les résultats doivent être comparés avant l'affichage.

Pour éviter de stocker en RAM l'intégralité des résultats (qui peuvent peser plusieurs Mo chacun), nous avons adopté l'approche suivante :

1. On calcule le résultat temporaire.
2. On extrait ses métriques (longueur via `strlen` et diversité via `count_digit`).
3. On libère immédiatement la mémoire du résultat.
4. On ne stocke que les métadonnées du ou des "meilleur(s) candidat(s)" trouvé(s) jusqu'ici (itération et valeur de départ) dans une liste chaînée.

Ce n'est qu'à la toute fin du programme que la fonction `solve` est appelée pour recalculer le résultat final des meilleurs candidats. Bien que cela implique de calculer deux fois le résultat gagnant, cela évite le crash par saturation de la RAM.

Structure de données

Les candidats ex-aequo sont stockés dans une liste chaînée simple :

Listing 2.3 – Structure de la liste chaînée Candidate

```
typedef struct Candidate {
    char *value;
    int iterations;
    struct Candidate *next;
} Candidate;
```

La variable `next` stocke l'adresse du candidat suivant (qui serait aussi "bon" que le(s) candidat(s) actuel(s)). Si un nouveau candidat est strictement meilleur que le(s) précédent(s), la liste existante est libérée grâce à la fonction `free_candidates`. Cette fonction assure qu'aucune fuite mémoire ne subsiste lors du nettoyage de la liste.

Listing 2.4 – Fonction de libération de la liste

```

void free_candidates(Candidate *first) {
    while (first != NULL) {
        Candidate *temp = first;
        first = first -> next;
        free(temp -> value);
        free(temp);
    }
}

```

2.4.2 Implémentation de l'option `--debug`

Contrairement aux autres fichiers du projet, le fichier `debug.h` ne contient pas de fonction classique, mais définit une macro préprocesseur. Ce choix d'implémentation est justifié par une contrainte d'optimisation des performances.

Si nous avons utilisé une fonction standard (ex : `void print_debug(...)`) définie dans un fichier `.c`, le programme aurait dû effectuer un appel de fonction systématique à chaque rencontre de l'instruction, impliquant :

- La sauvegarde du contexte d'exécution.
- Le passage des arguments.
- Le saut vers l'adresse mémoire de la fonction.

Et ce, même si le niveau de débogage était à 0.

En utilisant une macro `#define`, le préprocesseur effectue une substitution de texte avant la compilation. Le test conditionnel `if (debug_level >= level)` est ainsi injecté directement à l'endroit de l'appel.

Listing 2.5 – Macro

```

#define DEBUG(level, fmt, ...) \
    do { \
        if (debug_level >= level) { \
            fprintf(stderr, "[DEBUG:%d] " fmt "\n", level, ##__VA_ARGS__ \
                ); \
        } \
    } while (0)

```

Cela garantit que si le niveau de débogage n'est pas suffisant, le coût processeur est quasi nul (un simple test d'entier), et la lourde fonction `fprintf` n'est jamais appelée.

2.4.3 Écriture du résultat

Pour l'écriture dans le fichier de sortie, l'utilisation de `fprintf` s'est révélée problématique pour les très grandes chaînes. En effet, `fprintf` retourne un `int`, limité à $2^{31} - 1$. Lors du dépassement de cette limite, la fonction échoue.

Nous avons donc remplacé cet appel par `fwrite`, qui gère des flux de données binaires et utilise des types non signés (`size_t`), contournant ainsi la limitation de taille.

2.5 Gestion des LEDs

L'interaction avec les LEDs passe par le fichier `led.c`. Il est important de noter que notre montage utilise une configuration en anode commune. Cela implique une logique

inversée : mettre une broche à 0 (LOW) allume la LED, et la mettre à 1 (HIGH) l'éteint. C'est pourquoi, pour obtenir du rouge, nous mettons le canal Rouge à 0 et les autres à 1 :

Listing 2.6 – Activation de la LED Rouge

```
void setRedLed(void) {  
    setColor(0, 1, 1);  
}
```

La fonction **clearLed()** est systématiquement appelée en fin de programme pour libérer les lignes GPIO proprement.

Les LEDs permettent de garder informé l'utilisateur de l'état du processus :

- **Bleu / Cyan** : Alternance à chaque itération afin de visualiser l'exécution du programme.
- **Vert** : Fin de programme avec succès (return 0).
- **Rouge** : Erreur critique (fichier introuvable, mémoire saturée, etc.).

Chapitre 3

Gestion de la mémoire

La gestion manuelle de la mémoire en langage C nous impose de garder une rigueur stricte tout au long du développement afin d'éviter les fuites mémoire qui, sur un système embarqué comme le Raspberry Pi, pourraient saturer la RAM et provoquer un crash du système.

Pour se faire, nous allons utiliser l'outil **Valgrind** afin de vérifier l'absence de fuites a posteriori. Mais premièrement, nous avons opté pour une approche préventive lors de la conception de notre code.

3.1 Approche préventive

- **Libération en cascade (Listes chaînées)** : Dans le mode `-only-longest`, nous utilisons une liste chaînée de structures `Candidate` afin de stocker les meilleurs candidats. Pour éviter les fuites lors du nettoyage de la liste, nous avons implémenté la fonction `free_candidates` (dans `main.c`). Celle-ci a pour but de parcourir la liste et libérer systématiquement deux éléments pour chaque maillon : d'abord la valeur dynamique (`temp->value`), puis le conteneur (`temp`), garantissant ainsi qu'aucun candidat orphelin n'existe.
- **Sécurisation des réallocations** : Dans la fonction `solve` (`calcul.c`) par exemple, nous utilisons une variable temporaire (`tmp`) pour récupérer le résultat de `realloc`. Cette pratique empêche de perdre l'adresse du bloc mémoire original en cas d'échec de l'allocation (ce qui créerait une fuite irrécupérable).

Listing 3.1 – Réallocation mémoire

```
char *tmp = realloc(new_value, needed_alloc);
if (tmp == NULL) {
    free(current_value);
    free(new_value);
    return NULL;
}
```

- **Gestion des chemins d'erreur** : Une source fréquente de fuites est l'interruption du programme (ex : ligne vide, erreur de format). Dans notre fonction `run`, chaque chemin d'erreur appelle explicitement `free_candidates(candidates_list)` avant de retourner le code d'erreur 1.

3.2 Validation avec Valgrind

Pour confirmer l'efficacité de cette stratégie, nous avons soumis le programme à l'outil d'analyse dynamique **Valgrind** sur le Raspberry Pi. L'exécution se fait avec les drapeaux suivants :

```
valgrind --leak-check=full --show-leak-kinds=all --track-origins=yes \  
./main [arguments]
```

Nous avons testé trois scénarios distincts que nous allons vous présenter ci-dessous.

Scénario 1 : Exécution nominale

Ce test correspond au cas standard (calcul et affichage direct). Il valide que les buffers alloués pour la lecture du fichier (`getline/fgets`) et la génération de la suite (`solve`) sont correctement libérés à chaque itération.

Scénario 2 : Gestion de liste complexe (`--only-longest`)

Ce scénario est le plus critique car il implique la création, la manipulation et la suppression d'une liste chaînée. Le test a été réalisé avec un fichier générant de nombreux candidats ex-aequo suivis de purges. Le rapport confirme que la fonction `free_candidates` a correctement parcouru toute la chaîne pour tout libérer, et que le recalcul final (double appel à `solve`) n'a pas laissé de résidus.

Scénario 3 : Simulation d'erreur (Input invalide)

Nous avons injecté un fichier contenant des lignes vides et des formats incorrects pour déclencher les conditions d'erreur du `main.c`. Ce résultat prouve que même lors d'une sortie prématurée (via `return 1`), la libération de la mémoire est correctement effectuée.

3.3 Conclusion sur la fiabilité

L'absence totale d'erreurs ("0 errors from 0 contexts") et la confirmation que "All heap blocks were freed" dans tous les scénarios démontrent que notre implémentation est correcte d'un point vue fuite mémoire. Le programme peut donc s'exécuter sur de longues périodes sans risque de saturation progressive de la RAM.

Implémentation hardware

Le cahier des charges nous demandait d'avoir un retour visuel de l'état du programme (exécution en cours, succès ou échec). Pour répondre à cette demande, nous avons conçu un circuit basé sur des diodes (LEDs) contrôlées par les ports GPIO du Raspberry Pi.

4.1 Description du montage électronique

Le composant qui était initialement prévu s'avère être une LED RGB à quatre broches. Cependant, suite à une défaillance matérielle constatée sur le canal rouge de ce même composant, nous avons dû modifier légèrement notre architecture hardware en ajoutant une LED rouge indépendante. Le circuit final est donc composé des éléments suivants :

- 1 LED RGB (Canaux Vert et Bleu utilisés) ;
- 1 LED Rouge standard ;
- 3 Résistances de protection de $220\ \Omega$;
- 1 Breadboard ainsi que plusieurs cables

La Figure 4.1 ci-dessous illustre le schéma de câblage réalisé.

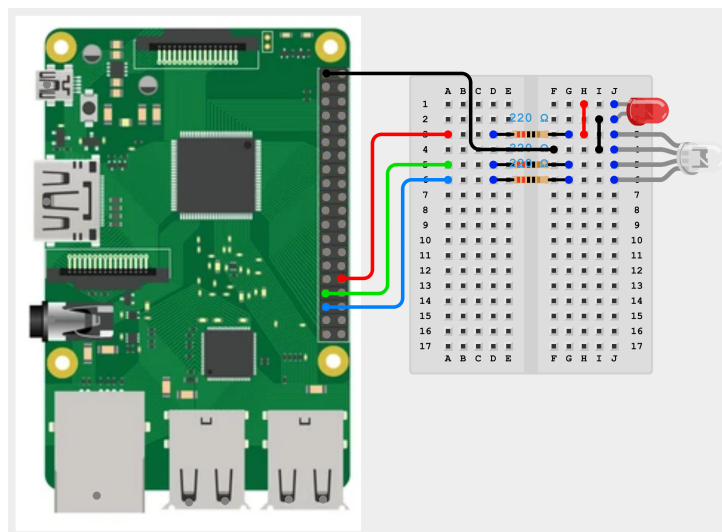


FIGURE 4.1 – Schéma de câblage : LED RGB (Anode Commune) et LED Rouge additionnelle.

4.2 Justification technique et contrôle

4.2.1 Configuration en anode commune

La LED RGB utilisée est de type Anode Commune. Cela signifie que la broche commune (la plus longue) est reliée à la tension d'alimentation positive (3.3V via la broche physique 1 du Raspberry Pi).

Cette configuration inverse ainsi la logique de programmation habituelle :

- **LED allumée** : Le GPIO est mis à l'état bas (0V). Le courant circule de l'alimentation vers le GPIO.
- **LED éteinte** : Le GPIO est mis à l'état haut (3.3V). La différence de potentiel est nulle alors le courant ne circule donc pas.

Nous avons fait le choix d'appliquer cette même logique pour la LED rouge additionnelle afin de garder une certaine cohérence, son anode est reliée au 3.3V, tandis que sa cathode est pilotée par le GPIO via une résistance.

4.2.2 Dimensionnement des résistances

Afin de préserver l'intégrité des ports GPIO de notre Raspberry Pi, nous avons placé des résistances de $R = 220\Omega$ en série avec chaque cathode. Nous pouvons, dès lors, valider ce choix en appliquant simplement la loi d'Ohm $I = \frac{U_{source} - U_{LED}}{R}$. En considérant une tension source de 3.3V et une tension de seuil typique (V_d) de 2V pour le rouge et 3V pour le bleu et le vert, nous obtenons les résultats suivant :

$$I_{rouge} = \frac{3.3V - 2V}{220\Omega} \approx 5.9mA \quad (4.1)$$

$$I_{vert/bleu} = \frac{3.3V - 3V}{220\Omega} \approx 1.36mA \quad (4.2)$$

Ces valeurs sont bien inférieures à la limite maximale de courant supportée par un pin GPIO (16mA), garantissant ainsi la sécurité du matériel.

4.3 Correspondance des broches

Le raccordement logique au sein de notre programme C, via la librairie `libgpiod`, suit la table de correspondance suivante (Table 4.1) :

Fonction / Couleur	Composant	Broche Physique	GPIO
Alimentation (+)	Commun	Pin 1	3.3V
Vert	LED RGB	Pin 33	GPIO 13
Bleu	LED RGB	Pin 35	GPIO 19
Rouge	LED Rouge Add.	Pin 32	GPIO 12

TABLE 4.1 – GPIOs utilisés pour le contrôle visuel

Chapitre 5

Évaluation quantitative et analyse des performances

Pour valider la pertinence de notre implémentation de la suite de Conway, nous avons réalisé un benchmark comparatif en conditions réelles directement sur le Raspberry Pi. Dans la suite, nous détaillerons donc le protocole de test, les différentes spécificités de l'environnement et finalement l'analyse des résultats que nous obtenons.

5.1 Environnement et protocole de test

5.1.1 Environnement

Le Raspberry Pi impose différentes contraintes en termes de puissance de calcul, de mémoire et de vitesse de lecture et d'écriture. Nous avons donc effectués les tests sur la configuration suivante :

- **Hardware** : Raspberry Pi 5.
- **Processeur** : Broadcom BCM2712 2.4GHz quad-core 64-bit Arm Cortex-A76 CPU.
- **Stockage** : Carte micro-SD 32Go.
- **Système d'exploitation** : Linux.
- **Environnement Python** : Python 3 a été installé ainsi que l'utilitaire `time`.

5.1.2 Méthodologie de reproduction

Afin de garantir la bonne reproductibilité de notre benchmark par quiconque, nous avons suivi le protocole suivant :

1. Les deux programmes (binaire C compilé avec `gcc` et le script Python) ont été placés dans un même répertoire sur la carte SD, bien que chaque programme fonctionne de manière indépendante.
2. Le programme C a été nettoyé de la partie concernant les LEDs, afin que les deux codes puissent accomplir exactement les mêmes objectifs sans autres possibles perturbations.
3. Aucun processus d'arrière-plan consommateur n'est exécuté durant les tests.

4. Un script d'automatisation lance séquentiellement les itérations de 1 à 70, en exécutant d'abord le programme C, puis le programme Python, pour chaque itération, en partant de la valeur initiale 1.
5. La lecture et l'écriture se fait grâce à des fichiers d'input et d'output via les options `-input` et `-output`.
6. Mesure du Temps : Utilisation du temps réel ressenti via la librairie standard.
7. Mesure de la RAM : Utilisation de la commande système `/usr/bin/time -f %M` pour capturer le pic de mémoire.

5.2 Analyse des résultats

5.2.1 Comparaison des temps d'exécution (Speedup)

La Figure 5.1 présente l'évolution du temps de calcul sur le Raspberry Pi en fonction du nombre d'itérations.

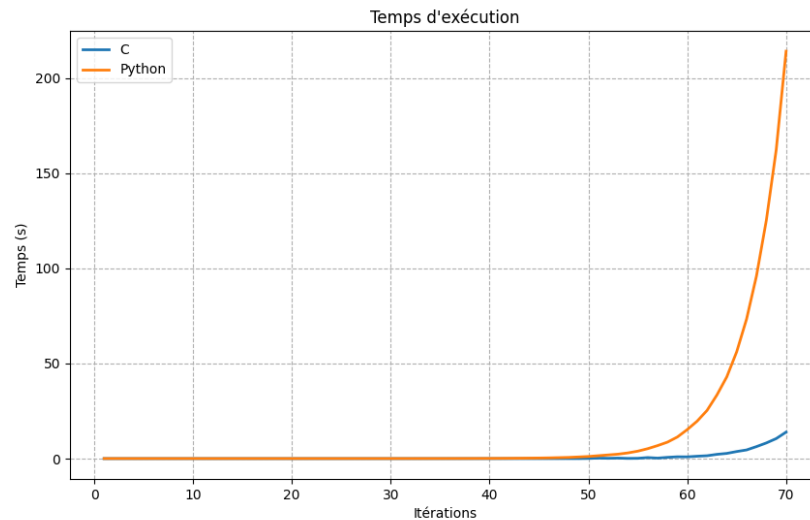


FIGURE 5.1 – Temps d'exécution sur Raspberry Pi (C vs Python).

À la 70^{ème} itération, le programme C termine en **13.94 secondes**, tandis que l'implémentation Python nécessite **214.14 secondes**. Le facteur d'accélération *Speedup* observé atteint ainsi **x15.4**. Le C est nettement plus rapide que Python avec un débit de ≈ 16 Mo/s à la 70^{ème} itération.

Analyse de la variation du Speedup

Une observation notable est la stabilisation du facteur d'accélération (*Speedup*) autour de x15 sur le Raspberry Pi, alors que d'après d'autres tests effectués directement sur un PC plus puissant et sous Windows nous atteignons un Speedup de x30. Nous pouvons expliquer ce phénomène par le déplacement de ce que l'on peut appeler un "goulot d'étranglement".

Bien que les opérations algorithmiques s'effectuent exclusivement avec la RAM, le programme doit, à la fin, écrire le résultat dans un fichier `output.txt` sur la carte SD. La

phase de calcul en C est très rapide, tandis que la phase d'écriture sur la carte micro-SD du Raspberry Pi est une opération plutôt lente. Etant donné que le code C est très rapide, ce temps d'écriture représente une proportion assez importante du temps total d'exécution, ce qui a pour effet de réduire mathématiquement le ratio de performance apparente. Cette latence agit comme un temps incompressible qui est ajouté aux deux programmes (C et Python).

5.2.2 Comparaison de l'utilisation de la mémoire (RAM)

La gestion de la mémoire est le point critique sur un système embarqué. La Figure 5.2 illustre la consommation mémoire maximale atteinte pour chaque itération.

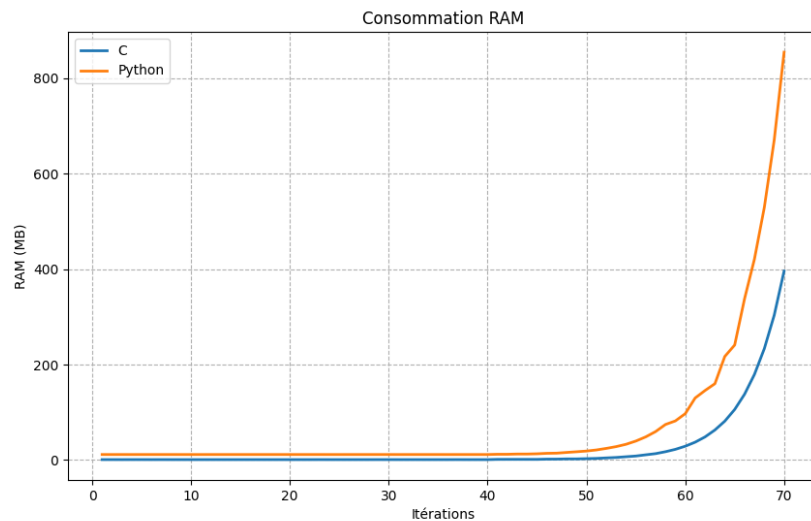


FIGURE 5.2 – Consommation RAM sur Raspberry Pi.

1. Utilisation initiale : Le programme C démarre avec une empreinte minime de 1.1 MB, correspondant au chargement des bibliothèques Linux. En revanche, Python mobilise immédiatement 11.7 MB pour charger sa machine virtuelle, ce qui représente un coût fixe important sur un système disposant de peu de RAM.

2. Efficacité : À la 70^{ème} itération :

- **C (396 MB) :** La mémoire est allouée essentiellement pour stocker la chaîne de caractères.
- **Python (854 MB) :** La consommation explose et approche 1Go. Le ratio est de **2.2x**.

Cette différence s'explique notamment par la structure des objets Python qui ajoute des métadonnées lourdes à chaque chaîne, contrairement à la gestion "brute" et optimisée des pointeurs en C.

5.2.3 Données détaillées

Le tableau ci-dessous (Figure 5.3) rassemble les métriques exactes relevées sur le Raspberry Pi. On y constate que le C maintient un débit d'écriture disque impressionnant (≈ 16 Mo/s) malgré les limitations de la carte SD.

Benchmark C vs Python									
Iter	Longueur	Temps C (s)	Temps Py (s)	Speedup (x)	Débit C (char/s)	Débit Py (char/s)	RAM C (MB)	RAM Py (MB)	Ratio RAM (x)
1	2	0.0015	0.0424	28.4	1,341	47	1.1	11.7	11.0
2	2	0.0016	0.0401	25.6	1,277	50	1.1	11.7	11.0
3	4	0.0016	0.0424	26.8	2,528	94	1.0	11.7	11.5
4	6	0.0016	0.0413	26.5	3,851	145	1.1	11.7	11.0
5	6	0.0016	0.0401	25.4	3,803	150	1.1	11.7	11.0
6	8	0.0015	0.0403	26.8	5,324	199	1.0	11.7	11.5
7	10	0.0015	0.0405	27.5	6,810	247	1.1	11.7	11.0
8	14	0.0015	0.0403	27.1	9,404	347	1.0	11.7	11.5
9	20	0.0015	0.0403	27.3	13,561	496	1.0	11.7	11.5
10	26	0.0015	0.0404	26.8	17,250	644	1.0	11.7	11.5
15	102	0.0015	0.0404	27.4	69,059	2,523	1.0	11.7	11.5
20	408	0.0015	0.0406	26.6	267,512	10,045	1.0	11.7	11.5
25	1,540	0.0015	0.0417	27.1	1,001,275	36,968	1.0	11.7	11.5
30	5,808	0.0016	0.0453	27.5	3,523,361	128,296	1.1	11.7	11.0
35	21,890	0.0020	0.0590	28.9	10,709,590	370,783	1.0	11.7	11.5
40	82,350	0.0036	0.1113	30.6	22,638,850	739,889	1.0	11.7	11.5
45	310,036	0.0091	0.3102	34.3	34,252,212	999,478	1.5	13.2	8.7
50	1,166,642	0.0957	1.1230	11.7	12,190,492	1,038,846	3.0	18.9	6.3
55	4,391,702	0.1641	3.9254	23.9	26,764,308	1,118,786	8.5	39.9	4.7
60	16,530,884	0.9251	15.2598	16.5	17,869,320	1,083,298	29.1	97.2	3.3
61	21,549,544	1.2566	19.6946	15.7	17,149,203	1,094,187	37.5	130.1	3.5
62	28,091,184	1.5088	25.3133	16.8	18,618,373	1,109,740	48.5	145.8	3.0
63	36,619,162	2.2472	33.4424	14.9	16,295,588	1,094,992	63.1	160.0	2.5
64	47,736,936	2.7428	42.9724	15.7	17,404,466	1,110,873	81.6	217.1	2.7
65	62,226,614	3.7226	56.1291	15.1	16,715,824	1,108,634	106.0	241.0	2.3
66	81,117,366	4.5572	73.3375	16.1	17,799,715	1,106,083	138.0	338.0	2.4
67	105,745,224	6.3093	95.9415	15.2	16,760,143	1,102,184	179.5	421.1	2.3
68	137,842,560	8.2342	125.2424	15.2	16,740,180	1,100,606	233.5	529.4	2.3
69	179,691,598	10.5259	162.3239	15.4	17,071,395	1,106,994	303.5	670.5	2.2
70	234,241,786	13.9387	214.1382	15.4	16,805,119	1,093,881	396.0	854.4	2.2

FIGURE 5.3 – Données brutes du benchmark Raspberry Pi.

5.3 Conclusion du benchmark

L'exécution sur Raspberry Pi valide définitivement le choix du langage C pour ce projet. Alors que le script Python accapare près de 1Go et monopolise le processeur pendant plus de 3 minutes 30, le programme C réalise la tâche en moins de 14 secondes avec une empreinte mémoire deux fois moindre.

Sur un système embarqué où l'énergie et la mémoire sont des ressources critiques, l'implémentation C n'est pas seulement une optimisation, c'est une nécessité pour un tel projet.

Chapitre 6

Conclusion

Au terme de ce projet, nous avons validé la conception d'un programme en C robuste, capable de générer la suite de Conway tout en contournant les limitations techniques inhérentes à l'écriture de grands volumes de données et à la gestion dynamique de la mémoire. L'architecture mise en place, couplée à un interfaçage matériel via des GPIOs, assure un traitement fluide et configurable, validant notre maîtrise du pipeline logiciel complet sur une plateforme embarquée.

L'évaluation quantitative confirme sans équivoque la supériorité de notre solution : l'implémentation en C offre un gain de performance (Speedup) moyen de x15 et divise par 2,2 la consommation mémoire par rapport à Python lors des itérations avancées. Ces résultats démontrent que l'usage d'un langage compilé, gérant finement les ressources, est non seulement une optimisation mais une nécessité pour garantir la viabilité de tâches intensives sur un Raspberry Pi.